Sentiri RAG Evaluation Does the Chatbot Retrieve the Right Evidence?

Retrieval & faithfulness measurement for the autism-clinic knowledge chatbot

Nik Abdul Aqmal Bin Zunaidi

2026-11-07

Table of contents

Introduction **2**
The pipeline under evaluation . . . 2
The full strategy of building this RAG pipeline . . . 3
Requirements that drive RAG evaluation . . . 5

The gold set **6**

Retrieval evaluation **7**
The headline finding: the reranker is not earning its latency . . . 7
Improvements, in evidence order . . . 8
Adversarial questions: retrieval cannot detect them . . . 8
ADD NEW RAG gold question . . . 10

Generation evaluation **18**

Failure analysis: the worst five **18**

Limitations **19**

Conclusion **19**

Daily Task

7 Nov 2026

- ☒ Finish evaluating rag retrieval results
- ☒ Prove that the description in adversarial column still not determine wether LLM will answer or not

tomorrow task

- ☐ Finish feeding our gold_set_rag question with much more challenging question **sophisticated question**
- ☐ Swap Embedding models to BGE-m3 to see wether is it any performance improvement

Introduction

The question this page answers: does Sentiri's RAG chatbot retrieve the *right* evidence and answer *from* it — measured, not assumed?

The model evaluation page defends model choices (XLM-R vs. alternatives, BERTopic vs. alternatives) with labeled-data comparison. The RAG pipeline cannot be defended that way: there is no "test split" for retrieval. The defense here is **measurement against a hand-built gold set**: retrieval ranked metrics, generation faithfulness, and failure analysis.

The pipeline under evaluation

```
question → embed (MiniLM multilingual)
         → ChromaDB cosine top-20            dense recall (cheap, broad)
         → cross-encoder rerank → top-5      precision (slow, sharp)
         → grounded LLM answer + citations   generation (Groq)
```

Two stages fail in different ways: retrieval misses the evidence,

generation ignores or overreaches it: so each is evaluated separately.

According to our pipeline we use **dense embeddings setup and NOT BM25 setup** why ? lets dive into it

Firstly, we need to understand where and how to use Best Match 25 ? in our architecture ?

```
question → tokenize (lowercase, Sastrawi stem)    query must see the
                                                   same preprocessing as
                                                   the index, or scores die
         → BM25 score vs inverted index    lexical recall (cheap, broad)
              (term freq × idf ÷ doc length)
         → top-20 candidate chunk ids
         → cross-encoder rerank → top-5       precision (slow, sharp)
         → grounded LLM answer + citations    generation (Groq)


Index side (built once, offline):


chunks → tokenize (same pipeline!) → inverted index:
         term → [(chunk_id, tf, doc_len)]


And since the real-world version is usually hybrid, the full picture:


  == BELOW SHOWS THE DIFFERENCES BETWEEN USING DENSE EMBEDDINGS PIPELINE VS BM-
25 ==

                   embed → ChromaDB cosine top-20
question                                          → fuse (RRF) → top-20
                   tokenize → BM25 top-20                → rerank → top-5
                                                                  → LLM + citations
```

The full strategy of building this RAG pipeline

```
FILE: chat_analyzer/services/rag_service.py
Ported from PersonalRAG, one key change (multilingual embedding for Malay).

MODELS
  Embedding: paraphrase-multilingual-MiniLM-L12-v2 (SentenceTransformer, 384-
dim, lazy singleton)
  Reranker:  cross-encoder/ms-marco-MiniLM-L-6-v2 (CrossEncoder, lazy singleton)
  LLM:       Groq openai/gpt-oss-120b, temp 0.1, max_tokens 1024 (GROQ_API_KEY env-
only)

STORE
  ChromaDB PersistentClient, dir data/vector_store/
```

```
  Collection "treatment_documents", hnsw:space=cosine
  Chunk ids: md5(doc_title)[:8] + index → e.g. "9349f30a_3" (these are your gold-
set annotation ids)

CONFIG (hardcoded constants, not env)
  CHUNK_SIZE 1000 chars, CHUNK_OVERLAP 200
  TOP_K 5 (post-rerank), RETRIEVE_K 20 (dense recall), SCORE_THRESHOLD 0.2

INGEST (ingest_document, called by Celery on doctor upload)
  load_and_chunk: PyMuPDFLoader (PDF) / TextLoader (.txt/.md) →
    RecursiveCharacterTextSplitter(1000/200, default separators) →
    metadata {source, page, chunk_index}
  embed_and_store: batch encode → collection.add(ids, embeddings, documents,
    metadata {source, page, chunk_index, doc_title, document_id?})
  Returns IngestionResult {status: ready|failed, chunks_created, error}
  document_id links chunks → TreatmentDocument row → file_url for citations

QUERY (query(), called by chatbot view)
  1. retrieve(question, top_k=20): encode query → ChromaDB cosine query →
     cosine distance→similarity = (2-d)/2 → filter similarity < 0.2 → RetrievedChunk
  2. rerank(query, docs, top_k=5): CrossEncoder.predict on (query, content) pairs →
     sort desc by rerank_score → top 5
  3. build_context: numbered [Document i - title, page (relevance: X%)] headers
  4. build_sources: {title, page, score, preview[:200], file_url} -
     single TreatmentDocument query (no N+1)
  5. calculate_confidence: sigmoid(max rerank score)
  6. generate_answer: Malay system prompt (grounded-only, refuse if insufficient,
     match answer language to question) + context + question → Groq
  Returns RAGResponse {answer, sources, confidence, context?}

UTILITIES
  get_collection_stats(): chunk count for dashboard
  delete_document_chunks(doc_title): where={doc_title} filter → delete

KNOWN STATE
  Collection: 14 chunks / 3 PDFs (annotated gold ids from col.get())
  Reranker: caught degrading hit@5 1.00→0.64 in eval, currently still in the
    pipeline path (fix-or-remove verdict pending - BGE-m3 one-line swap is the fix)
```

Note

BGE-M3 (Chen et al. 2024)? what is it ?

1. Paper: Chen, Xiao, Zhang, Luo, Lian, Liu (2024). "BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation." arXiv:2402.03216
2. Model card: https://huggingface.co/BAAI/bge-m3
3. Code: https://github.com/FlagOpen/FlagEmbedding (FlagEmbedding/BGE_M3)

   An **embedding model** from BAAI Beijing Ai Academy m3 means = three multi's

- Multi-Functionality: one model, three retrieval modes — dense, sparse (lexical), multi-vector (ColBERT-style) — trained via unified fine-tuning with self-knowledge distillation from all three. The sparse output is a vocab-sized vector, mostly zeros, weight per token present in the text — same family as BM25/uniCOIL/SPLADE but learned.
- Multi-Linguality: 100+ working languages.
- Multi-Granularity: inputs from short sentences up to 8192 tokens.
- Concrete spec: 1024-dim embeddings (vs MiniLM's 384), max_length 8192, architecture = xlm-roberta-large extended to 8192 and further pretrained with RetroMAE, then contrastive learning (bge-m3-unsupervised), then unified fine-tuning.

But first we will use standard ms-marco-MiniLM-L-6-v2

Requirements that drive RAG evaluation

#	Requirement	Consequence for evaluation
R1	Parent questions in Malay / English / rojak; documents in Malay & English	Gold set must include code-switched queries, not just clean English
R2	Medical-adjacent domain: hallucinated therapy advice is the worst failure mode	Faithfulness (answer grounded in retrieved chunks) is the priority generation metric
R3	Therapists must verify every claim against the source	Citations must point at real, retrievable chunks: retrieval quality is upstream of trust

#	Requirement	Consequence for evaluation
R4	The system must know when it does not know	Refusal accuracy on out-of-domain questions is measured explicitly
R5	Production configuration: retrieve-20 → rerank-5, chunk 1000/200, cosine	Config constants are evaluated as-is, then challenged in the failure analysis

What is NOT evaluated here: latency, concurrent load, embedding-model comparison (single-model pipeline), or Celery ingestion behavior. Scope is retrieval quality + generation faithfulness on the treatment-document knowledge base.

The gold set

`gold_set_rag.json` — N questions hand-annotated with the chunk IDs that contain the correct evidence (`doc_title` + `chunk_index` addressing, the same keys stored in ChromaDB metadata).

Design rules:

- Real parent-style questions, drawn from what the clinic chatbot actually receives — including Malay-English code-switching (R1).
- 2–3 difficulty tiers: direct lookup, multi-hop (answer spans two chunks), and paraphrase (question shares no keywords with the evidence).
- **Adversarial out-of-domain questions** — plausible-sounding questions the documents cannot answer. The correct behavior is the refusal line ("Saya tidak mempunyai maklumat…"), and these questions measure whether the system knows when it does not know (R4).

```
import json
from pathlib import Path

if Path("gold_set_rag.json").exists():
    with open("gold_set_rag.json") as f:
        gold = json.load(f)
    print(f"{len(gold)} questions loaded - composition table pending annotation.")
else:
    print("gold_set_rag.json not built yet - this cell will fill in once the")
    print("gold set exists (see The gold set section for the design rules).")
```

```
13 questions loaded - composition table pending annotation.
```

Retrieval evaluation

Ranked-retrieval metrics, computed twice — **dense-only (top-20) vs. dense+rerank (top-5)** — on the same gold set, the same fairness rule as the model-evaluation page: only the pipeline stage changes.

Metric	Question it answers	Formula
Hit@k	Is *any* gold chunk in the top-k?	mean over questions of 1[found]
Recall@k	Are *all* gold chunks in the top-k?	mean of |top-k gold| / |gold|
MRR	How high does the *first* gold chunk rank?	mean of 1/rank_first_gold (0 if never)
Precision@k	How much noise feeds the LLM?	|top-k gold| / k (lower bound: gold annotation not exhaustive)

The dense vs. rerank delta is the ablation that matters: it is the evidence for whether the cross-encoder earns its latency (R5).

The headline finding: the reranker is not earning its latency

Dense retrieval is excellent: **hit@20 is perfect** — for all 11 real questions, the embedding model places at least one gold chunk inside the top-20 candidates. The funnel mouth never misses.

The cross-encoder rerank then **reduces** quality: MRR drops from 0.56 to 0.44, hit@5 falls 0.82→0.73 (3 of 11 questions lose gold from the top-5), and the stricter **recall@5** — all gold chunks inside the window, not just one — falls 0.77→0.59. The failure mode is *displacement*: dense retrieval hands the reranker gold in strong positions (rank 1, 3, 4) and the cross-encoder demotes it below rank 5. Worst case — *"why does my autistic child only want to eat one brand of food"* — gold sat at **dense rank 1** and rerank pushed it out of the top 5.

The failures are not language-bound (2 English, 2 Malay), so "English-trained reranker (ms-marco-MiniLM) struggles with Malay" is only part of the diagnosis. The reranker is also *credited* with two rank-3 → rank-1 rescues — it wins some questions and loses others, net negative on this gold set.

Improvements, in evidence order

1. **Equal-k comparison first**: score dense top-5 vs rerank top-5 on the same footing before any removal decision (the harness supports this with two extra metric calls).
2. **Swap the cross-encoder** for a multilingual reranker (BGE-m3 family) and re-run — the eval harness makes this a one-line change plus one 37-second re-run. The numbers decide.
3. **Gate rerank by question language** if the swap does not close the gap.

Adversarial questions: retrieval cannot detect them

Both adversarial questions retrieve chunks with similarity **0.79–0.85** — nearly as high as real questions. With `SCORE_THRESHOLD = 0.2`, the retrieval layer passes them through confidently; the only barrier to a hallucinated answer (invented therapy costs) is the LLM choosing to refuse. Refusal accuracy is therefore a generation-layer metric, not a retrieval one.

```
import json
import pandas as pd

with open("rag_eval_results.json") as f:
    res = json.load(f)
agg = res["aggregate"]

# Collect-then-score: the equal-k dense metrics are recomputed from the
# saved evidence (dense_ranked_ids per record) - no pipeline re-run needed.
def hit_at_k(ranked_ids: list[str], gold_ids: list[str], k: int) -> int:
    return 1 if set(gold_ids) & set(ranked_ids[:k]) else 0

def precision_at_k(ranked_ids: list[str], gold_ids: list[str], k: int) -> float:
    return len(set(ranked_ids[:k]) & set(gold_ids)) / k

def recall_at_k(ranked_ids: list[str], gold_ids: list[str], k: int) -> float:
    """Strict companion to hit@k: ALL gold must be inside top-k, not just one."""
    return len(set(ranked_ids[:k]) & set(gold_ids)) / len(gold_ids)

real = [r for r in res["records"] if not r["adversarial"]]
dense_hit5 = sum(hit_at_k(r["dense_ranked_ids"], r["gold_chunk_ids"], 5) for r in real) / len
dense_p5 = sum(precision_at_k(r["dense_ranked_ids"], r["gold_chunk_ids"], 5) for r in real) /
dense_rec5 = sum(recall_at_k(r["dense_ranked_ids"], r["gold_chunk_ids"], 5) for r in real) /
rerank_rec5 = sum(recall_at_k(r["rerank_ranked_ids"], r["gold_chunk_ids"], 5) for r in real)
gold_lost_rerank = sum(
    1 for r in real
```

```
    if recall_at_k(r["dense_ranked_ids"], r["gold_chunk_ids"], 5) == 1.0
    and recall_at_k(r["rerank_ranked_ids"], r["gold_chunk_ids"], 5) < 1.0
)

# Every number followed by its interpretation - the table argues,
# not just reports.
df = pd.DataFrame([
    {"Metric": "Hit@20", "Dense only": agg["hit@20"], "Dense + rerank": None,
     "Reading": "perfect - gold reaches the top-20 for all 11 real questions"},
    {"Metric": "Hit@5", "Dense only": agg["hit@5"], "Dense + rerank": agg["hit@5_rerank"],
     "Reading": "3/11 questions lose gold from the top-5 after reranking"},
    {"Metric": "Recall@5", "Dense only": dense_rec5, "Dense + rerank": rerank_rec5,
     "Reading": f"strict: ALL gold inside top-5 - rerank evicts the sole gold chunk on {gold_
    {"Metric": "MRR", "Dense only": agg["mrr_dense"], "Dense + rerank": agg["mrr_rerank"],
     "Reading": "rerank demotes gold on average - MRR drops"},
    {"Metric": "Precision@5", "Dense only": agg["precision@5"], "Dense + rerank": agg["precis
     "Reading": "precision@k is capped at 1/k"},
])

styler = (df.style
    .hide(axis="index")
    .format(precision=2, na_rep="-")
    .set_properties(subset=["Reading"], **{"text-align": "left", "white-space": "normal"})
)
styler
```

Table 1: Retrieval metrics — dense-only vs dense+rerank, equal-k where it matt

Metric	Dense only	Dense + rerank	Reading
Hit@20	1.00	—	perfect — gold reaches the top-20 for all 11 real questions
Hit@5	0.82	0.73	3/11 questions lose gold from the top-5 after reranking
Recall@5	0.77	0.59	strict: ALL gold inside top-5 — rerank evicts the sole gold chunk
MRR	0.60	0.42	rerank demotes gold on average — MRR drops
Precision@5	0.16	0.15	precision@k is capped at 1/k

"**precision@5** is low because each question has only ~1-2 gold chunks, so the metric is capped at $|\text{gold}|/5 = 0.2$ it measures how much of the top-5 window is gold not wether retrieval found the right chunk ('thats hit@5'). observed 0.164 approximately around 0.82 x 0.2, consistent with hit@5 = 0.82"

ADD NEW RAG gold question

Our current rag gold questions

Table 2: Current gold question set (13 questions, 11 real + 2 a

	#	Question	Gold chunks	Lang	Diff
0	1	cara meningkatkan kemahiran bertutur anak autism?	7a2ee7cf_0, 7a2ee7cf_1	ms	easy
1	2	anak saya tak reti tunggu giliran bercakap, ma...	7a2ee7cf_1	rojak	mediun
2	3	what is PECS and how does it help non-verbal c...	7a2ee7cf_1	en	easy
3	4	anak saya belum boleh bercakap langsung, ada a...	7a2ee7cf_2	ms	mediun
4	5	apa aktiviti yang ibu bapa boleh buat di rumah...	7a2ee7cf_2	ms	mediun
5	6	berapa kerap anak autism perlu attend sesi ter...	7a2ee7cf_3	rojak	easy
6	7	kenapa anak autism saya suka menjerit bila nak...	7b6d878e_1, 7b6d878e_2	ms	hard
7	8	sebutan anak saya tak clear, apa latihan untuk...	7a2ee7cf_0, 7a2ee7cf_3	ms	mediun
8	9	berapa kos sesi terapi pertuturan di Malaysia?	— (refusal)	ms	na
9	10	anak saya ada tanda autism, umur berapa patluh...	— (refusal)	rojak	na
10	11	anak saya tolak sayur sebab tekstur, apa nak b...	a9aab0e9_1	ms	mediun
11	12	why does my autistic child only want to eat on...	a9aab0e9_1	en	mediun
12	13	berapa peratus kanak-kanak autism ada masalah ...	a9aab0e9_0	ms	easy

The results of our retrieval [click to see the result]

```
import json
import pandas as pd

with open("rag_eval_results.json") as f:
    results = json.load(f)

def rank_of(ids: list[str], gold: set[str]) -> str:
    return next((str(i+1) for i, cid in enumerate(ids) if cid in gold), "miss")

rows = []
for rec in results["records"]:
    if rec["adversarial"]:
        continue  # refusal questions: judged at generation, not retrieval
    gold = set(rec["gold_chunk_ids"])
    d_ids, r_ids = rec["dense_ranked_ids"], rec["rerank_ranked_ids"]
    rows.append({
        "#": rec["id"],
        "Question": rec["question"][:40] + "…",
        "Gold": ", ".join(rec["gold_chunk_ids"]),
        "Dense rank": rank_of(d_ids, gold),
        "Rerank rank": rank_of(r_ids, gold),
        "Dense@5": " " if set(d_ids[:5]) & gold else " ",
        "Rerank@5": " " if set(r_ids[:5]) & gold else " ",
    })

pd.DataFrame(rows)
```

Table 3: Per-question retrieval results - dense top-20 vs reranked top-5 vs gold

	#	Question	Gold	Dense rank	Rerank r
0	1	cara meningkatkan kemahiran bertutur ana…	7a2ee7cf_0, 7a2ee7cf_1	1	1
1	2	anak saya tak reti tunggu giliran bercak…	7a2ee7cf_1	3	1
2	3	what is PECS and how does it help non-ve…	7a2ee7cf_1	1	3
3	4	anak saya belum boleh bercakap langsung,…	7a2ee7cf_2	4	miss
4	5	apa aktiviti yang ibu bapa boleh buat di…	7a2ee7cf_2	5	miss
5	6	berapa kerap anak autism perlu attend se…	7a2ee7cf_3	1	2
6	7	kenapa anak autism saya suka menjerit bi…	7b6d878e_1, 7b6d878e_2	6	2
7	8	sebutan anak saya tak clear, apa latihan…	7a2ee7cf_0, 7a2ee7cf_3	7	4
8	11	anak saya tolak sayur sebab tekstur, apa…	a9aab0e9_1	2	2

9	12	why does my autistic child only want to …	a9aab0e9_1	1	miss
10	13	berapa peratus kanak-kanak autism ada ma…	a9aab0e9_0	1	2

```
print(f"dense rank and rerank rank ? \nit means for dense retrieval embeddings the first ch

print(f"\nbut how about the Adversarial question ?")
```

```
dense rank and rerank rank ?
it means for dense retrieval embeddings the first chunks ids in
what rank it shows up the highest

but how about the Adversarial question ?
```

> below are the test for our adversarial question <

```
rows = []
for rec in results["records"]:
    if not rec["adversarial"]:
        continue
    d_ids, d_scores = rec["dense_ranked_ids"], rec["dense_scores"]
    rows.append({
        "#": rec["id"],
        "Question": rec["question"][:45] + "…",
        "Chunks past threshold": len(d_ids),
        "Top chunk": d_ids[0] if d_ids else "none",
        "Top similarity": round(d_scores[0], 3) if d_scores else "-",
        "Top rerank score": round(rec["rerank_scores"][0], 3) if rec.get("rerank_scores") e
    })
pd.DataFrame(rows)
```

Table 4: Adversarial questions — what retrieval does when no evidence exists

	#	Question	Chunks past threshold	Top chunk	Top simil
0	9	berapa kos sesi terapi pertuturan di Malaysia…	20	7a2ee7cf_3	0.710
1	10	anak saya ada tanda autism, umur berapa patlu…	20	7a2ee7cf_3	0.846

Lets dive into this one by one col. ?

1. “Chunks pass threshold”=20

We asked for top-20 chunks, so all 20 chunks scored similarity >= 0.5, because our cutoff is 0.2, so all 20 survive.

==========

2. “Top chunk” = 7a2ee7cf_3

This is just the ID of the chunk that ranked #1 for this question. It’s name not a score yet. Which chunk is it?

7a2ee7cf_3 is the speech therapy doc, chunk 3 - the one containing “2-3 kali seminggu, 45min-1jam” (session frequency/duration)

So when a parent asks about PRICE, the #1 chunk retrieval grabs is the one about how OFTEN sessions happen.

Related topic, wrong content.>

3. “Top similarity” = 0.710

This is ChromaDB’s cosine similarity between the question’s embedding and that top chunk’s embedding,

converted from distance. It answers: “how confident does DENSE retrieval feel about this top chunk?” 0.710 on a 0-1 scale (roughly).

For context, REAL questions that got correctly answered

their top similarities are in the same ballpark (Q10 even scored 0.846!).

That’s the punchline of this column: the similarity score for an unanswerable question (0.71, 0.85)

is indistinguishable from the score for an answerable one.

The number LOOKS confident but means nothing: similarity measures topic closeness (“therapy”),

not answer presence (“price”). A similarity threshold can never separate these.

4. “Top rerank score” = 2.305 (Q9) vs -4.103 (Q10)

This is the cross-encoder’s score for the (question, top chunk) pair.

It answers a DIFFERENT question than similarity: “does this chunk actually ANSWER this question?”

the cross-encoder reads both texts together, so it grades relevance, not topic. Scale: unbounded, roughly anything positive = “this looks like an answer”, strongly negative = “this does not answer it.”

Q10: -4.1 → the reranker says loud and clear “this chunk does NOT answer ‘what age to start intervention’.” Correct judgment.

Q9: +2.3 → the reranker says “yes, this looks relevant to ‘how much do sessions cost’ ” — WRONG judgment, because frequency and cost are lexically close (“berapa kerap” vs “berapa kos”). The reranker got fooled on the near-miss.

BUT the real question is ? will LLM give answer for this question ?? [click here to see the results]

```
import sys, os
sys.path.insert(0, "/Users/apple/degree/finalyearengineredo")

# load env keys (GROQ_API_KEY) without hardcoding
with open("/Users/apple/degree/finalyearengineredo/.env") as f:
    for line in f:
        line = line.strip()
        if line and not line.startswith("#") and "=" in line:
            k, v = line.split("=", 1)
            os.environ.setdefault(k.strip(), v.strip())

os.environ["USE_SQLITE"] = "1"  # local dev db (no TreatmentDocument table)
os.environ.setdefault("DJANGO_SETTINGS_MODULE", "finalyearengineredo.settings")
import django; django.setup()

from chat_analyzer.services.rag_service import (
    retrieve, rerank, build_context, generate_answer,
)

adversarial = [
    (9,  "berapa kos sesi terapi pertuturan di Malaysia?"),
    (10, "anak saya ada tanda autism, umur berapa patluh start intervention?"),
]

results: list[dict] = []
for qid, q in adversarial:
    dense   = retrieve(q, top_k=20)          # 20 chunks, nothing filtered
    top5    = rerank(q, dense, top_k=5)      # reranker picks 5
    context = build_context(top5)            # these go to the LLM
    answer  = generate_answer(q, context)    # the moment of truth

    refused = "tidak mempunyai maklumat" in answer.lower()
    results.append({
        "#": qid, "question": q, "refused": refused, "answer": answer,
    })
    print(f"--- Q{qid}: {q}")
    print(f"    context chunks: {len(top5)}, top rerank: {top5[0]['rerank_score']:.2f}
    print(f"    REFUSED: {refused}")
    print(f"    ANSWER: {answer}")
    print()

refusal_rate = sum(r["refused"] for r in results) / len(results)
print(f"refusal accuracy: {refusal_rate:.0%} ({sum(r['refused'] for r in results)}/{le
```

```
--- Q9: berapa kos sesi terapi pertuturan di Malaysia?
    context chunks: 5, top rerank: 2.30
    REFUSED: True
    ANSWER: Saya tidak mempunyai maklumat yang cukup untuk menjawab soalan ini.

--- Q10: anak saya ada tanda autism, umur berapa patluh start intervention?
    context chunks: 5, top rerank: -4.10
    REFUSED: True
    ANSWER: Saya tidak mempunyai maklumat yang cukup untuk menjawab soalan ini. (Tiada

refusal accuracy: 100% (2/2)
```

What to add to our gold set of question ? [click here]

Current set: 13 questions (11 real + 2 adversarial). Dense retrieval is at ceiling on it — hit@20 = 1.00, so **no embedding change can show improvement on questions that are already all retrieved**. New questions must be designed to break the ceiling, each targeting a named failure mode:

1. Multi-hop (2 gold chunks, different docs) — the visible gap. Questions whose answer needs evidence from two documents, e.g. a feeding problem that is treated WITH a communication approach ("anak saya tolak makan sebab tak reti nak minta — apa nak buat?" → feeding chunk + AAC/PECS chunk). Current set has no question with gold chunks in two different docs. Tests: can retrieval surface BOTH, or does one topic dominate?

2. Hard paraphrase (zero lexical overlap). Q2 tested mild paraphrase ("tunggu giliran" vs "gilir bercakap"). Next level: questions that share no words with the gold chunk — pure semantics ("anak saya buat hal bila tak dapat apa dia nak" for the screaming/tantrum chunk, where the doc says "menjerit" / "challenging behaviour"). Tests: how far can paraphrase go before dense retrieval breaks?

3. Cross-lingual, hard mode. Q3/Q12 are English→Malay-doc, but on easy topics. Add one where the English question uses vocabulary that never appears in the Malay chunk. Tests: the embedding space's cross-lingual depth, which is exactly what a BGE-m3 swap claims to improve.

4. Near-miss adversarial (the Q9 trap, weaponized). Current adversarial questions fail cleanly. Add ones where retrieval DOES find topic-adjacent content that plausibly-but-wrongly answers (e.g. a medication-dosage or therapy-qualification question near the session-frequency chunk). Tests: refusal under real temptation — the generation-layer failure case, not the easy one.

5. Expanded refusal set (adversarial, easy mode, quantity). 5–8 more unanswerable questions (prices, locations, doctor names, legal, vaccine) so refusal accuracy becomes a real n, not n=2.

Plan: ~8–10 new questions total, old 13 kept (regression check — pipeline runs in ~37s). Annotate gold chunk ids by hand from col.get() as before, freeze the set, rerun. THEN the BGE-m3 embedding swap has measurable room to help — and if it doesn't, that's a finding too.
Order matters: gold set first (the instrument), model swap second (the treatment). Otherwise the swap verdict is meaningless.

Generation evaluation

Faithfulness (LLM-as-judge). For each question, the judge receives the retrieved context and the produced answer — not the system prompt — and lists every factual claim, marking each as supported or unsupported by the context. Faithfulness = supported / total claims.

Refusal accuracy. On adversarial questions: correct only if the answer refuses AND does not answer anyway. The inverse failure — refusing valid questions (false refusal) — is reported too, because a system that says "I don't know" to everything is useless even if it never hallucinates.

```
print("Pending generation eval - faithfulness + refusal table will render here.")
```

```
Pending generation eval - faithfulness + refusal table will render here.
```

Failure analysis: the worst five

The five questions where the pipeline ranked gold evidence lowest — each with a diagnosis: **chunking miss** (evidence split across chunks), **embedding miss** (semantics too far), **lexical trap** (keyword-matching retrieved the wrong section), or **rerank displacement** (cross-encoder demoted the gold chunk). This section is the input to the next pipeline iteration — the same role the trap-subset analysis plays on the model-evaluation page.

```
print("Pending eval results - worst-five table will render here.")
```

```
Pending eval results - worst-five table will render here.
```

Limitations

- Precision@k is a **lower bound**: gold annotation marks the best chunks, not every relevant one.
- The gold set is small (N questions) and built by one annotator — the project author. Reporting exact numbers with honest confidence beats a larger set built carelessly.
- Faithfulness uses an LLM as judge; judge errors are possible and the judge scores are not human-verified.
- Retrieval eval measures ranking quality on a frozen collection state; production ingests new documents continuously.

Conclusion

To be written once the numbers exist — one-breath answer format, same as the model-evaluation page: what was measured, what the pipeline does well, where it fails, and what the next iteration changes.

Chen, Jianlv, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024. *BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation*. https://arxiv.org/abs/2402.03216.